

딥러닝 기반 수식 이미지 **LaTeX** 변환 OCR 시스템 구현

Kuggle 2차 프로젝트

최흥준 | 엄창현 | 송채영 | 이지유 | 장재원

INDEX

01

개요

02

전처리

03

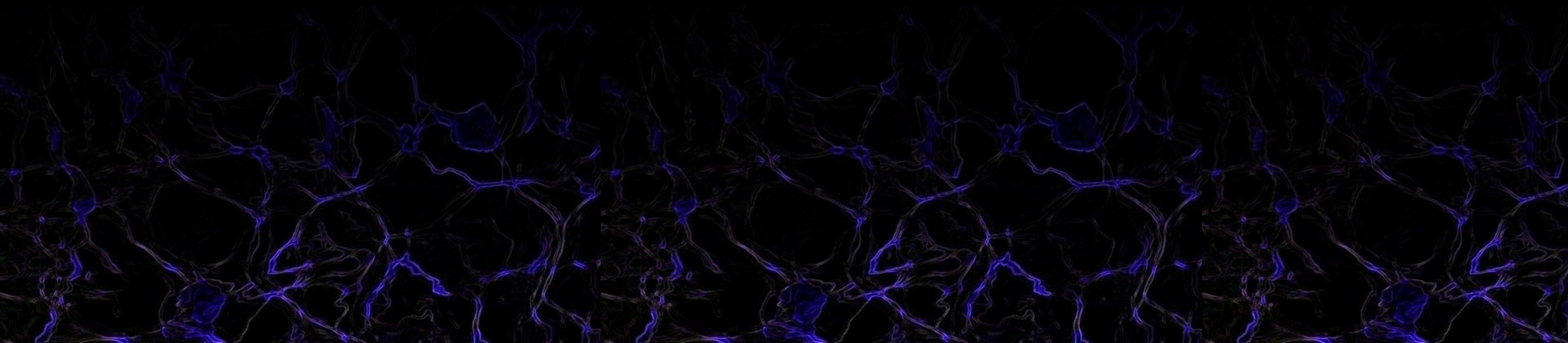
모델링

04

분석 결과

05

최종 결론

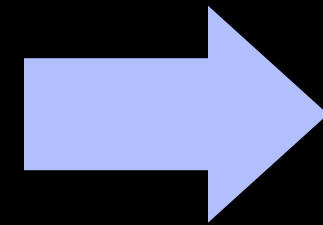


Outline _ 개요

이 프로젝트의 목적은 **LaTeX 코드를 자동 생성하는 딥러닝 모델을 개발하는 것**이에요

Problem

$$x^2 + y^2 = z^2$$



$$x^2 + y^2 = z^2$$

직접 입력하는 것의 불편함
→ 일반 OCR의 한계

문서 자동화 가능

Solution



수식 이미지를 입력받아 대응되는 LaTeX 코드를 자동 생성하는 딥러닝 모델 개발

Kaggle의 'IM2LATEX-100K' 데이터셋을 사용했어요

'IM2LATEX-100K'



- image 10만 개
- im2latex_formulas.norm.csv : formulas(LaTeX)
- im2latex_train.csv : formula(LaTeX), image(.png)
- im2latex_validate.csv
- im2latex_test.csv

이미지 이진화를 통해 수식 이미지의 배경 정보를 제거하고 수식 구조를 강조했어요

Otsu Thresholding

```
_, bw = cv2.threshold(  
    gray,  
    0,  
    255,  
    cv2.THRESH_BINARY  
    + cv2.THRESH_OTSU  
)
```

VS

Adaptive Thresholding

```
blurred = cv2.GaussianBlur(gray, (1, 1), 0)  
  
bw = cv2.adaptiveThreshold(  
    blurred,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11,  
    20  
)
```

* Pixel > T 는 White로, Pixel < T 는 Black으로

이미지 이진화를 통해 수식 이미지의 배경 정보를 제거하고 수식 구조를 강조했어요

train: preprocessing comparison (sample dir: /content/dataset/data/images)

Original (grayscale)

$$V^{(2)}(\xi_1,\xi_2)=\frac{V(\xi_1)V(\xi_2)-\hat{g}(V_2(\xi_1)+V_2(\xi_2))}{1+c-\hat{g}(\xi_1+\xi_2)},$$

Binarized (Otsu)

$$V^{(2)}(\xi_1,\xi_2)=\frac{V(\xi_1)V(\xi_2)-\hat{g}(V_2(\xi_1)+V_2(\xi_2))}{1+c-\hat{g}(\xi_1+\xi_2)},$$

Binarized (Adaptive)

$$V^{(2)}(\xi_1,\xi_2)=\frac{V(\xi_1)V(\xi_2)-\hat{g}(V_2(\xi_1)+V_2(\xi_2))}{1+c-\hat{g}(\xi_1+\xi_2)},$$

Binarized (Adaptive)

$$-\int [dg]\chi_R^*(g)V(g)\,.$$

Original (grayscale)

$$H\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle=E\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle,\\U_k\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle=\left(n_k+p_k^{\rm min}\right)\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle,\quad k=1,\ldots,N-1,$$

Binarized (Otsu)

$$H\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle=E\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle,\\U_k\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle=\left(n_k+p_k^{\rm min}\right)\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle,\quad k=1,\ldots,N-1,$$

Binarized (Adaptive)

$$H\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle=E\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle,\\U_k\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle=\left(n_k+p_k^{\rm min}\right)\left|\begin{array}{c}E,[n] \\[q]\end{array}\right\rangle,\quad k=1,\ldots,N-1,$$

Binarized (Adaptive)

$$=\lambda\left(\frac{1}{2}\frac{d\tilde{\phi}}{d\lambda}+\tilde{\phi}\right)$$

Original (grayscale)

$$\langle \Phi_1,\Phi_2\rangle=\int d^4x\,\sqrt{g}\,\Phi_1^+\Phi_2\quad.$$

Binarized (Otsu)

$$\langle \Phi_1,\Phi_2\rangle=\int d^4x\,\sqrt{g}\,\Phi_1^+\Phi_2\quad.$$

Binarized (Adaptive)

$$\langle \Phi_1,\Phi_2\rangle=\int d^4x\,\sqrt{g}\,\Phi_1^+\Phi_2\quad.$$

Binarized (Adaptive)

$$\frac{1}{\kappa+1}=\frac{1}{z^{2\kappa+2}}\sum_{k=0}^{\infty}\left(\begin{array}{c}\kappa+k \\k\end{array}\right)\left(\frac{t}{z}\right)^{2k}$$

Original (grayscale)

$$S^c=\frac{1}{\pi}\int d^2\sigma\left(c^+\partial_-b_++c^-\partial_+b_-\right)$$

Binarized (Otsu)

$$S^c=\frac{1}{\pi}\int d^2\sigma\left(c^+\partial_-b_++c^-\partial_+b_-\right)$$

Binarized (Adaptive)

$$S^c=\frac{1}{\pi}\int d^2\sigma\left(c^+\partial_-b_++c^-\partial_+b_-\right)$$

Original (grayscale)

$$\lambda_R\,=\,-\int [dg]\chi_R^*(g)V(g)\,.$$

Binarized (Otsu)

$$\lambda_R\,=\,-\int [dg]\chi_R^*(g)V(g)\,.$$

Binarized (Adaptive)

$$\lambda_R\,=\,-\int [dg]\chi_R^*(g)V(g)\,.$$

Binarized (Adaptive)

$$\frac{1}{2}\left(\hat{T}_{\hat{a}\hat{z}\hat{b}}+\hat{T}_{\hat{b}\hat{z}\hat{a}}-\hat{T}_{\hat{a}\hat{b}\hat{z}}\right),$$

Original (grayscale)

$$\phi=\lambda\left(\frac{1}{2}\frac{d\tilde{\phi}}{d\lambda}+\tilde{\phi}\right)$$

Binarized (Otsu)

$$\phi=\lambda\left(\frac{1}{2}\frac{d\tilde{\phi}}{d\lambda}+\tilde{\phi}\right)$$

Binarized (Adaptive)

$$\phi=\lambda\left(\frac{1}{2}\frac{d\tilde{\phi}}{d\lambda}+\tilde{\phi}\right)$$

Binarized (Adaptive)

$$x\sqrt{g}\left(W-\Phi R+XR^2+YR_{ij}R^{ij}+ZR_{ijkl}R^{ijkl}\right),$$

Original (grayscale)

$$\frac{1}{(z^2-t^2)^{\kappa+1}}=\frac{1}{z^{2\kappa+2}}\sum_{k=0}^{\infty}\left(\begin{array}{c}\kappa+k \\k\end{array}\right)\left(\frac{t}{z}\right)^{2k}\,.$$

Binarized (Otsu)

$$\frac{1}{(z^2-t^2)^{\kappa+1}}=\frac{1}{z^{2\kappa+2}}\sum_{k=0}^{\infty}\left(\begin{array}{c}\kappa+k \\k\end{array}\right)\left(\frac{t}{z}\right)^{2k}\,.$$

Binarized (Adaptive)

$$\frac{1}{(z^2-t^2)^{\kappa+1}}=\frac{1}{z^{2\kappa+2}}\sum_{k=0}^{\infty}\left(\begin{array}{c}\kappa+k \\k\end{array}\right)\left(\frac{t}{z}\right)^{2k}$$

Binarized (Adaptive)

$$e\rlap{\mathcal{A}}^c-m)\psi^c=\bar{\psi}(i\rlap{\mathcal{D}}-e\rlap{\mathcal{A}}-m)\psi$$

이미지 리사이징과 패딩 작업을 진행했어요

```
scale = min(  
    target_h / float(h),  
    target_w / float(w)  
)
```

```
new_h = max(1, int(round(h * scale)))  
new_w = max(1, int(round(w * scale)))
```

```
resized = cv2.resize(  
    work,  
    (new_w, new_h),  
    interpolation=cv2.INTER_AREA  
)
```

```
canvas = np.full(  
    (target_h, target_w),  
    pad_value,  
    dtype=np.float32  
)  
  
y0 = (target_h - new_h) // 2  
x0 = (target_w - new_w) // 2  
  
canvas[y0:y1, x0:x1] = resized
```

리사이징

- 수식 이미지는 크기와 비율이 모두 다르기 때문에 모델 입력 크기를 통일
- 왜곡을 방지하고 수식 구조를 보존하기 위해 이미지 가로세로 비율을 유지한 상태로 리사이징

패딩

- 모든 이미지 크기 통일
- 중앙 정렬
- 입력 형태 고정

이미지를 정규화하고 텐서로 변환했어요

```
work = gray.astype(np.float32) / 255.0
```

```
Normalize(  
    mean=(0.5,),  
    std=(0.5,) )
```

정규화

- 모델이 안정적으로 학습할 수 있도록 픽셀 범위를 0~1로 정규화
- Gradient 안정화 및 학습 성능 향상을 위해 -1~1 사이로 다시 정규화

```
img_t = torch.from_numpy(img).float()
```

텐서 변환

- PyTorch 입력 형식으로 변환
- (H, W) -> (1, H, W)

이진화 → 리사이징 → 패딩 → 정규화 → 텐서 변환

토큰화를 통해 텍스트 전처리를 진행했어요

```
LATEX_TOKEN_RE = re.compile(
    r"\\[a-zA-Z]+\*?" # \frac, \sqrt*, etc.
    r"|\\[^\a-zA-Z\s]" # \, \; \ (single-char control seq after \)
    r"|\{|\}"
    r"|[\d-9]+"
    r"|[\a-zA-Z]+"
    r"|[\^\\s\\\\\{\\}\a-zA-Z\d-9]"
)

def tokenize_latex(formula: str) -> list[str]:
    """Tokenize a LaTeX string; commands like \frac are single tokens."""
    s = formula.strip()
    if not s:
        return []
    tokens: list[str] = []
    for m in LATEX_TOKEN_RE.finditer(s):
        tok = m.group(0)
        if tok.strip() == "":
            continue
        tokens.append(tok)
    return tokens
```

토큰화

- 정규표현식을 이용해 LaTeX 수식을 토큰 단위로 분리
- \명령어, 중괄호, 숫자, 그 외

어휘 사전을 생성하고 정수 인코딩을 진행했어요

```
token_to_idx = {}

for i, sp in enumerate(
    SPECIAL_TOKENS
):
    token_to_idx[sp] = i

next_id = len(SPECIAL_TOKENS)

for tok, c in sorted(
    counts.items(),
    key=lambda x: (-x[1], x[0])
):
    token_to_idx[tok] = next_id
    next_id += 1
```

어휘 사전 생성

- 토큰의 등장 빈도를 계산하여 자주 등장하는 토큰 우선 학습
- 각 토큰에 정수 ID 부여

```
ids = [
    self.stoi.get(
        t,
        self.unk_id
    )
    for t in tokenize_latex(formula)
]

ids = [
    self.sos_id
] + ids + [
    self.eos_id
]
```

정수 인코딩

- 각 토큰의 ID로 변환
- 특수 토큰 추가

토큰화 → 어휘 사전 매핑 → 특수 토큰 추가 → 정수 시퀀스 변환

드라이브를 연결하고 데이터셋 저장 폴더를 생성해 압축을 해제했어요

```
# 드라이브 연결
from google.colab import drive
drive.mount('/content/drive')
```

구글 드라이브 가상 디렉토리 연동 (drive.mount)

- 구글 코랩 가상 환경과 구글 드라이브 계정을 네트워크로 연결

```
# /content/drive/MyDrive/LaTeX_Project/에 있는 zip 파일을
# /content/dataset 폴더에 압축 해제
!mkdir -p /content/dataset
!unzip -q "/content/drive/MyDrive/LaTeX_Project/im2latex.zip" -d /content/dataset
```

데이터셋 저장 폴더 생성 및 고속 압축 해제

- 데이터셋 폴더 생성
- 구글 드라이브에 저장된 im2latex.zip 파일 압축 해제

드라이브를 연결하고 데이터셋 저장 폴더를 생성해 압축을 해제했어요

```
import torch
```

```
from im2latex_data_pipeline import (  
    build_vocab_from_train,  
    LatexDataset,  
    create_data_loader,  
    gray_normalize_neg1_pos1  
)
```

```
IMAGE_MODE = "binarized_adaptive" # 3개다 돌려봐야함.  
root_path = "/content/dataset"  
# Vocab 생성  
vocab = build_vocab_from_train(root_path)  
# 이미지 정규화 설정  
postprocess = gray_normalize_neg1_pos1()
```

전처리 설정

- **im2latex_data_pipeline** 파일 import
- **IMAGE_MODE** : 3가지 모드를 각각 교차 검증
- **vocab** 및 **postprocess** 정의 (단어장 빌드 및 정규화):
 - 학습 데이터셋 내부의 수식 라벨을 기반으로 고유한 단어장을 생성
 - `gray_normalize_neg1_pos1()` 함수를 통해 이미지 픽셀 값 정규화

드라이브를 연결하고 데이터셋 저장 폴더를 생성해 압축을 해제했어요

Dataset 인스턴스 생성

```
train_dataset = LatexDataset(  
    root=root_path,  
    split="train",  
    vocab=vocab,  
    image_mode=IMAGE_MODE,  
    image_postprocess=postprocess  
)
```

```
val_dataset = LatexDataset(  
    root=root_path,  
    split="val",  
    vocab=vocab,  
    image_mode=IMAGE_MODE,  
    image_postprocess=postprocess  
)
```

DataLoader 생성

```
train_loader = create_dataloader(train_dataset, batch_size=32, shuffle=True)  
val_loader = create_dataloader(val_dataset, batch_size=32, shuffle=False)
```

```
print(f"데이터 로더 준비 완료! (학습용 배치 수: {len(train_loader)})")
```

```
데이터 로더 준비 완료! (학습용 배치 수: 2353)
```

Dataset 생성

• train_dataset & val_dataset 생성

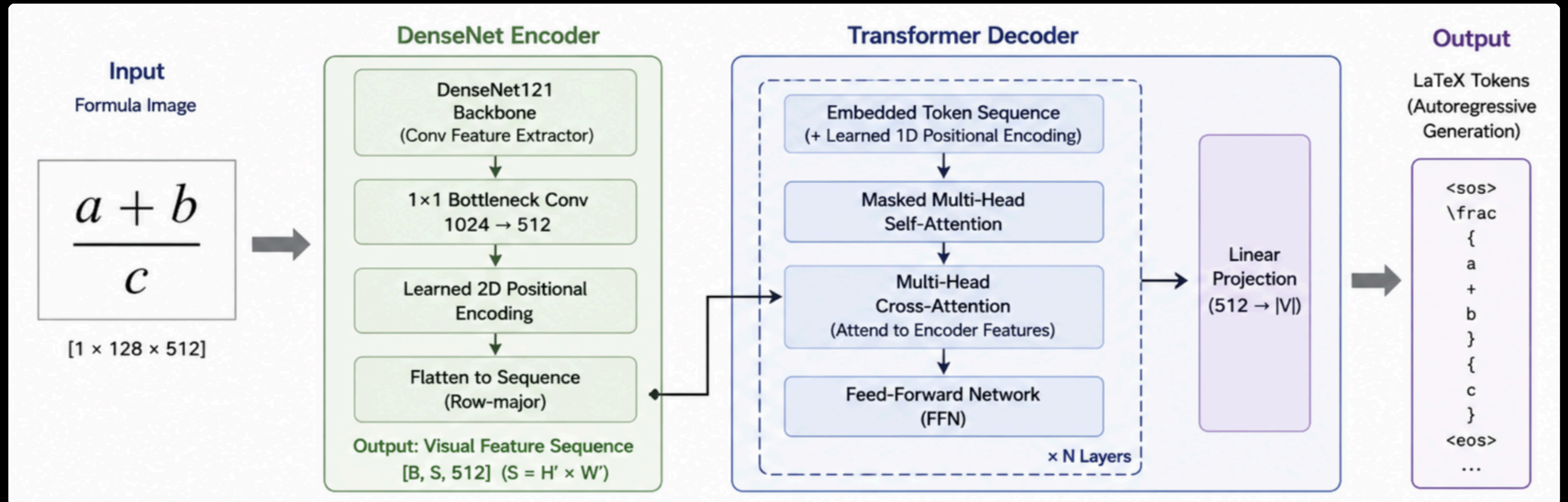
- LatexDataset 클래스를 활용해 학습용과 검증용 인스턴스를 각각 분리하여 생성
- 동일한 단어장, 이미지 처리 모드, 정규화 프로세스를 통일성 있게 주입

DataLoader 생성

• train_loader & val_loader (배치 데이터 로더 구축)

- 대용량 데이터를 모델이 효율적으로 학습할 수 있도록 create_dataloader 함수를 통해 미니배치 형태로 묶어주는 과정
- batch_size=32: 한 번의 학습 단계마다 32개의 이미지-수식 쌍 데이터를 묶어 모델에 공급하도록 배치 크기를 설정
- shuffle=True (Train 적용): 학습용 데이터는 에폭마다 무작위로 섞어 모델이 데이터의 순서적 특징을 외우는 과적합 현상 방지
- shuffle=False (Val 적용): 검증용 데이터는 모델의 순수한 예측 성능만을 객관적으로 평가해야 하므로 순서를 섞지 않고 일관되게 유지

모델링 아키텍처를 도식화했어요



인코더를 통해 입력된 수식 이미지를 받아 딥러닝 백본을 통해 시각적 특징을 추출하고,
디코더가 읽을 수 있는 시퀀스 형태로 변환했어요

1 models.py _ 인코더

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    if x.dim() != 4 or x.size(1) != 1:
        raise ValueError(
            f"Expected input shape [B, 1, H, W], got {tuple(x.shape)}."
        )

    batch_size = x.size(0)

    features = self._extract_backbone_features(x)

    projected = self.bottleneck(features)

    encoded = self.pos_encoding(projected)

    seq_len = encoded.size(2) * encoded.size(3)
    sequence = encoded.flatten(2).transpose(1, 2).contiguous()

    assert sequence.shape == (batch_size, seq_len, self.hidden_dim), (
        f"Unexpected output shape {tuple(sequence.shape)}; "
        f"expected ({batch_size}, {seq_len}, {self.hidden_dim})."
    )

    return sequence
```

1. 백본 특징 추출 (DenseNet-121)

- 이미지의 시각적 특징들을 추출하여 고차원 특징 맵 생성
- 출력 크기: [Batch _ Size, 1024, H', W']

2. Bottleneck 프로젝션 (차원 축소)

- 1024개의 채널을 디코더가 쓸 hidden _ dim 크기로 압축
- 출력 크기: [Batch _ Size, hidden _ dim, H', W']

인코더를 통해 입력된 수식 이미지를 받아 딥러닝 백본을 통해 시각적 특징을 추출하고,
디코더가 읽을 수 있는 시퀀스 형태로 변환했어요

1 models.py _ 인코더

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    if x.dim() != 4 or x.size(1) != 1:
        raise ValueError(
            f"Expected input shape [B, 1, H, W], got {tuple(x.shape)}."
        )

    batch_size = x.size(0)

    features = self._extract_backbone_features(x)

    projected = self.bottleneck(features)

    encoded = self.pos_encoding(projected)

    seq_len = encoded.size(2) * encoded.size(3)
    sequence = encoded.flatten(2).transpose(1, 2).contiguous()

    assert sequence.shape == (batch_size, seq_len, self.hidden_dim), (
        f"Unexpected output shape {tuple(sequence.shape)}; "
        f"expected ({batch_size}, {seq_len}, {self.hidden_dim})."
    )

    return sequence
```

3. 2D 위치 인코딩 (Learned 위치 정보 주입)

- 압축된 특징 맵에 가로/세로 위치 정보(바이어스)를 더함
- 출력 크기: [Batch _ Size, hidden _ dim, H', W']

4. 1D 시퀀스 평탄화 (Flatten & Transpose)

- 2D 이미지 형태를 1D 문장(시퀀스) 형태로 변환
- 출력 크기: [Batch _ Size, Seq _ Len, hidden _ dim]

디코더를 통해 인코더가 추출한 이미지 특징을 바탕으로, LaTeX 문자(토큰 시퀀스)을 한 글자씩 예측, 생성

2 models.py _ 디코더

```
def forward(
    self,
    tgt: torch.Tensor,
    memory: torch.Tensor,
    tgt_key_padding_mask: torch.Tensor | None = None,
    memory_key_padding_mask: torch.Tensor | None = None,
) -> torch.Tensor:
```

```
    batch_size, tgt_len = tgt.shape
    device = tgt.device
```

```
    tgt_emb = self.token_embedding(tgt) * math.sqrt(self.hidden_dim)
```

```
    tgt_emb = self.pos_encoding(tgt_emb)
```

```
    tgt_mask = self.generate_causal_mask(tgt_len, device)
```

1. 토큰 임베딩

- 입력된 수식 단어 ID들을 hidden_dim 크기의 연속적인 고차원 벡터 공간으로 맵핑
- 학습 안정성을 위해 채널 차원의 제곱근(sqrt)을 곱해 스케일링
- 출력 크기: [Batch_Size, Tgt_Seq_Len, hidden_dim]

2. 1D 위치 인코딩 주입

- 수식 내 단어들의 배열 순번(기호의 위치) 정보를 바이어스로 주입
- 출력 크기: [Batch_Size, Tgt_Seq_Len, hidden_dim]

3. Causal Mask 셀프 어텐션 (문맥 파악)

- 학습 시 디코더가 예측할 때 뒤에 올 정답 단어를 미리 보지 못하도록 미래 시점을 차단
- 마스크를 씌워 오직 현재와 과거 시점의 토큰들만 참조하여 예측
- 출력 크기: [Batch_Size, Tgt_Seq_Len, hidden_dim]

디코더를 통해 인코더가 추출한 이미지 특징을 바탕으로, LaTeX 문자(토큰 시퀀스)을 한 글자씩 예측, 생성

2 models.py _ 디코더

```
decoder_out = self.transformer_decoder(  
    tgt=tgt_emb,  
    memory=memory,  
    tgt_mask=tgt_mask,  
    tgt_key_padding_mask=tgt_key_padding_mask,  
    memory_key_padding_mask=memory_key_padding_mask,  
)
```

```
logits = self.output_projection(decoder_out)
```

```
assert logits.shape == (batch_size, tgt_len, self.vocab_size), (  
    f"Unexpected logits shape {tuple(logits.shape)}; "  
    f"expected ({batch_size}, {tgt_len}, {self.vocab_size})."  
)
```

```
return logits
```

4. Cross 어텐션 (옮겨 적기)

- 문맥 벡터(3단계 결과)와 인코더로부터 얻은 정보(수식 이미지 특징 정보)를 대조
- 이미지의 특정 부분의 시각적 단서를 가져와 정확한 기호 매칭
- 출력 크기: [Batch _ Size, Tgt _ Seq _ Len, hidden _ dim]

5. Output 프로젝트션

- 디코더를 통과하며 계산된 차원 벡터를 전체 LaTeX 단어 사전 크기로 확장
- 각 토큰 위치마다 사전에 정의된 LaTeX 기호 중 어떤 기호가 올 확률이 가장 높은지 점수표를 반환
- 출력 크기: [Batch _ Size, Tgt _ Seq _ Len, vocab _ size]

trainer.py의 데이터를 전처리하고 배치 처리를 진행했어요

1 trainer.py _ 데이터 전처리 및 배치 처리

```
def _unpack_batch(self, batch: Batch) -> Tuple[torch.Tensor, torch.Tensor]:  
  
    if isinstance(batch, dict):  
        images = batch[self.image_key]  
        tokens = batch[self.tokens_key]  
    elif isinstance(batch, (tuple, list)):  
        if len(batch) < 2:  
            raise ValueError(  
                "Tuple/list batches must be (images, tokens, ...)."  
            )  
        images, tokens = batch[0], batch[1]  
    else:  
        raise TypeError(  
            f"Unsupported batch type {type(batch)!r}. "  
            "Use a dict or (images, tokens) tuple."  
        )  
    return images, tokens
```

_ unpack _ batch

- 다양한 형태의 입력 데이터를 하나로 통일
- 데이터 로더(DataLoader)로부터 전달받은 배치 데이터의 자료 구조(dict, tuple, list 등)를 식별
- 모델 학습에 필수적인 이미지 텐서와 정답 수식 토큰 시퀀스를 안전하게 분리 및 추출

trainer.py의 데이터를 전처리하고 배치 처리를 진행했어요

1 trainer.py _ 데이터 전처리 및 배치 처리

```
@staticmethod
def _shift_for_teacher_forcing(
    tokens: torch.Tensor,
    padding_idx: int,
) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:

    tgt_in = tokens[:, :-1]
    tgt_out = tokens[:, 1:]
    tgt_key_padding_mask = tgt_in == padding_idx
    return tgt_in, tgt_out, tgt_key_padding_mask
```

_ shift _ for _ teacher _ forcing

- 수식 예측 학습을 위한 정답 가공
- tgt_in : 디코더 입력값 생성
 - 맨 마지막 토큰만 제거하고 슬라이싱
 - 다음 토큰을 예측하기 위해 참조할 수 있는 문장
- tgt_out : 예측 타겟(정답) 생성
 - 맨 첫 번째 토큰([SOS])만 제거하고 슬라이싱 (모델이 예측하는 대상 아님)
- tgt_key_padding_mask : 패딩 마스크 생성
 - 빈칸 채우기용 패딩 토큰(PAD)이 있는 위치는 True, 의미 있는 수식 토큰이 있는 위치는 False로 매핑된 부울(Bool) 행렬 생성

trainer.py의 순방향 연산을 수행하고 손실을 계산했어요

2 trainer.py _ 순방향 연산 및 손실 계산

```
def _forward(
    self,
    images: torch.Tensor,
    tgt_in: torch.Tensor,
    tgt_key_padding_mask: torch.Tensor,
) -> torch.Tensor:

    memory = self.encoder(images)
    logits = self.decoder(
        tgt=tgt_in,
        memory=memory,
        tgt_key_padding_mask=tgt_key_padding_mask,
    )
    return logits
```

_forward

- 인코더, 디코더 결합 (-> 순방향 연산 수행)
- 입력 이미지(원본 데이터)는 인코더에 통과시켜 연산
 - 최종 결과물: memory (시각 특징 맵)
- 가공된 텍스트 시퀀스(tgt_in)는 디코더에 통과시켜 연산
 - 최종 결과물: logits
 - (최종 확률 점수 행렬 : 출력층에서 계산된 가공되지 않은 점수)

trainer.py의 순방향 연산을 수행하고 손실을 계산했어요

2 trainer.py _ 순방향 연산 및 손실 계산

```
def _compute_loss(  
    self,  
    logits: torch.Tensor,  
    tgt_out: torch.Tensor,  
    ) -> torch.Tensor:  
  
    return self.criterion(logits.transpose(1, 2), tgt_out)
```

_compute_loss

- 예측 결과와 정답 간의 오차(Loss) 산출
- 모델의 예측값인 logits와 실제 정답 시퀀스인 tgt_out을 비교하여, 모델이 수식을 문맥과 시각 정보에 맞게 잘 맞추었는지 수치화(Cross-Entropy Loss)

trainer.py의 순평가 지표를 계산해 성능을 평가했어요

3 trainer.py _ 평가 지표 계산

```
for i in range(batch_size):  
    pred_str = self.decode_fn(pred_ids[i]) or ""  
    target_str = self.decode_fn(target_ids[i]) or ""  
  
    if not target_str and not pred_str:  
        cer_scores.append(0.0)  
    elif not target_str or not pred_str:  
        cer_scores.append(1.0)  
    else:  
        cer_scores.append(float(jiwer.cer(target_str, pred_str)))  
  
    ref_tokens = target_str.split()  
    hyp_tokens = pred_str.split()
```

compute_sequence_metrics : 성능 평가

1. 토큰 ID의 문자열 복원 (Decoding)

- 숫자 형태로 되어 있던 모델의 예측값(pred_ids[i])과 정답(target_ids[i])을 인간이 읽을 수 있는 LaTeX 수식 문자열(pred_str, target_str)로 복원

2. CER (자문 오차율) 계산

- 복원된 두 문자열을 글자(Character) 단위로 비교하여 오차율을 산출

3. 문자열의 토큰 배열 변환

- 복원된 LaTeX 수식 문자열(pred_str, target_str)을 공백(.split()) 기준으로 분리
- 기호 및 단어 단위의 1차원 토큰 배열(ref_tokens, hyp_tokens)로 가공

trainer.py의 순평가 지표를 계산해 성능을 평가했어요

3 trainer.py _ 평가 지표 계산

```
if not ref_tokens and not hyp_tokens:
    bleu_scores.append(1.0)
elif not ref_tokens or not hyp_tokens:
    bleu_scores.append(0.0)
else:
    bleu_scores.append(
        float(
            sentence_bleu(
                [ref_tokens],
                hyp_tokens,
                smoothing_function=smoothing,
            )
        )
    )

return {
    "cer": sum(cer_scores) / len(cer_scores),
    "bleu": sum(bleu_scores) / len(bleu_scores),
}
```

4. BLEU 점수 계산

- 분리된 두 토큰 배열을 n-gram 단위로 교차 대조하여 문맥적 유사도를 산출



CER (글자 단위 오차율)

BLEU (토큰 단위 문맥 유사도)

우수

0.0에 가까울수록 우수

1.0에 가까울수록 우수

척도

오타, 누락, 잘못된 기호 삽입이 얼마나 적은가

수식의 언어적 흐름과 결합 순서가 얼마나 정확한가

환경을 설정하고 경로를 정의해 모델을 학습했어요

1 모델 학습 _ 환경 설정 및 경로 정의

환경 설정

```
from trainer import Trainer, build_criterion

from models import FormulaDenseNetEncoder, FormulaTransformerDecoder
```

- 외부 파일 (trainer.py)에서 모델 학습을 총괄하는 Trainer 클래스와 손실함수를 생성하는 build_criterion 함수를 불러옴
- model 모듈에서 수식 이미지의 특징을 추출할 인코더와 텍스트를 생성할 디코더를 불러옴

경로 정의

```
VOCAB_SIZE = len(vocab)
HIDDEN_DIM = 512
NAME = "hongjun"
CHECKPOINT_DIR = f"/content/drive/MyDrive/LaTeX_Project/checkpoints_{NAME}_{IMAGE_MODE}"
```

- 사전(vocab)에 등록된 토큰의 개수를 VOCAB_SIZE에 저장
- 인코더와 디코더가 공유하는 은닉 상태 벡터의 크기를 512로 설정
- 학습된 모델의 체크포인트를 저장할 디렉토리 경로 생성

모델을 생성하고 손실 함수 및 최적화를 설정했어요

2 모델 학습 _ 모델 생성, 손실 함수 및 최적화 설정

```
encoder = FormulaDenseNetEncoder(hidden_dim=HIDDEN_DIM, pretrained=True)
decoder = FormulaTransformerDecoder(vocab_size=VOCAB_SIZE, hidden_dim=HIDDEN_DIM)
```

모델 생성

- DenseNet 인코더 생성 : 일반 이미지로 미리 학습된 가중치(pretrained=True)를 사용해 학습 효율을 높임
- Transformer 디코더 생성

모델을 생성하고 손실 함수 및 최적화를 설정했어요

2 모델 학습 _ 모델 생성, 손실 함수 및 최적화 설정

```
# 손실 함수
criterion = build_criterion(padding_idx=vocab.pad_id, label_smoothing=0.1)
# 최적화
optimizer = torch.optim.AdamW(list(encoder.parameters()) + list(decoder.parameters()), lr=1e-4)
```

손실 함수 및 최적화 설정

- 손실 함수 정의 :
 - 패딩 토큰(vocab.pad_id)은 손실 계산에서 제외하고 과적합을 방지하기 위해 정답 확률을 약간 깎아주는 LabelSmoothing(0.1) 기법을 적용
- 최적화 알고리즘(AdamW) 정의 :
 - 인코더와 디코더의 모든 파라미터(parameters())를 하나의 리스트로 묶어 동시에 학습
 - 학습률(Learning Rate)은 $1e-4$ (=0.0001)로 설정

Trainer를 설정하고 학습을 실행했어요

3 모델 학습 _ Trainer 설정 및 학습 실행

```
# Trainer 설정
trainer = Trainer(
    encoder=encoder,
    decoder=decoder,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    criterion=criterion,
    padding_idx=vocab.pad_id,
    num_epochs=20,
    checkpoint_dir=CHECKPOINT_DIR,
    image_key="images",
    tokens_key="formula_ids",
    decode_fn=vocab.decode,
    compute_metrics=True # CER, BLEU 계산
)
```

Trainer 객체 생성 (학습 진행 자동화)

- train_loader : 학습 데이터를 배치 단위로 모델에 공급
- val_loader : 검증용 데이터를 배치 단위로 모델에 공급
- num_epochs : 데이터 셋 반복 학습 횟수
- image_key : 입력용 수식 이미지 데이터를 꺼내오는 키
- tokens_key : 정답 토큰 ID를 꺼내오는 키
- decode_fn : 숫자 ID를 문자열로 변환
- compute_metrics : CER과 BLEU 자동 계산

Trainer를 설정하고 학습을 실행했어요

3 모델 학습 _ Trainer 설정 및 학습 실행

```
checkpoint_path = CHECKPOINT_DIR + "/latest.pt"

if os.path.exists(checkpoint_path):
    last_completed_epoch = trainer.load_checkpoint(checkpoint_path)
    trainer.train(start_epoch=last_completed_epoch + 1)
else:
    trainer.train()
```

체크포인트 파일(latest.pt) 경로 지정

- 세이브 파일 역할
- 지정한 경로에 학습하던 체크포인트 파일이 존재한다면, 해당 파일의 내용을 불러옴
- 마지막으로 완료된 epoch의 다음 epoch부터 이어서 학습
- 지정한 경로에 체크포인트 파일이 없다면, epoch 0부터 학습

Trainer를 설정하고 학습을 실행했어요

```
Epoch [1/20] train_loss=2.2621 val_loss=1.5541 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [2/20] train_loss=1.4374 val_loss=1.2848 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [3/20] train_loss=1.2564 val_loss=1.1989 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [4/20] train_loss=1.1779 val_loss=1.1486 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [5/20] train_loss=1.1353 val_loss=1.1201 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [6/20] train_loss=1.1077 val_loss=1.1001 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [7/20] train_loss=1.0883 val_loss=1.1168 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [8/20] train_loss=1.0749 val_loss=1.0891 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [9/20] train_loss=1.0635 val_loss=1.0859 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [10/20] train_loss=1.0551 val_loss=1.0798 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [11/20] train_loss=1.0474 val_loss=1.0973 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [12/20] train_loss=1.0414 val_loss=1.0720 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [13/20] train_loss=1.0363 val_loss=1.0743 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [14/20] train_loss=1.0320 val_loss=1.0558 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [15/20] train_loss=1.0275 val_loss=1.0569 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [16/20] train_loss=1.0240 val_loss=1.1089 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [17/20] train_loss=1.0202 val_loss=1.0546 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
Epoch [18/20] train_loss=1.0171 val_loss=1.1348 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [19/20] train_loss=1.0146 val_loss=1.2812 lr=1.00e-04 CER=0.0000 BLEU=1.0000
Epoch [20/20] train_loss=1.0114 val_loss=1.0512 lr=1.00e-04 CER=0.0000 BLEU=1.0000 *best*
```

환경 및 하이퍼파라미터를 정리하여 모델링을 진행했어요

Dataset:

- Train 80000 (80%)
- Validation 10000 (10%)
- Test 10000 (10%)

Image : 1 x 128 x 512

Training:

- Batch size : 32
- Optimizer : AdamW
- Learning Rate : 1e-4
- Epochs : 20
- Loss Function : CrossEntropy
- Gradient Clipping : 1.0
- LR Scheduler : ReduceLROnPlateau

Encoder: DenseNet-121

- Output Channels : 1024 -> 512
- Output Sequence Shape : [B, S, 512]

Decoder: Transformer

- Hidden Dimension : 512
- Number of Layers : 6
- Number of Heads : 8
- Feed Forward Dimension : 2048

Evaluation Metrics

- CER
- BLEU

테스트 데이터셋 및 데이터로더를 구축하여 모델링을 평가했어요

```
test_dataset = LatexDataset(  
    root=DATASET_ROOT,  
    split="test",  
    vocab=vocab,  
    image_mode="original", # 3개다 돌려봐야함.  
    image_postprocess=gray_normalize_neg1_pos1(),  
)
```

```
test_loader = create_dataloader(  
    test_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=False,  
    num_workers=4,  
)
```

```
print(f" Total test samples: {len(test_dataset)}")  
print(f" Checkpoint: {BEST_CHECKPOINT}")
```

1. 테스트 데이터셋 정의 및 정규화 (LatexDataset)

- 10,284개의 독립된 검증 데이터를 로드하고 픽셀 값을 정규화

2. 고속 미니배치 스트림 가동 (create_dataloader)

- 배치 크기 16 및 CPU 코어 4개를 할당하여 병목 없이 순서대로 데이터를 처리

3. 검증 환경 최종 레포팅 (print)

- 최종 샘플 수(10284)와 탑재할 가중치 경로를 화면에 출력해 정상 작동 확인

최종 지표를 산출하고 출력하여 모델링을 평가했어요

```
metrics = run_evaluation(  
    checkpoint_path=BEST_CHECKPOINT,  
    test_loader=test_loader,  
    vocab=vocab,  
    encoder=encoder,  
    decoder=decoder,  
    hidden_dim=HIDDEN_DIM,  
    max_gen_len=256,  
    device=DEVICE,  
)
```

최종 지표 산출

1. 통합 평가 루틴 구동 (run_evaluation)

- 가중치(best.pt)와 테스트 로더를 결합하여 10,284개 전체 샘플에 대한 일괄 추론 개시

2. 추론 제약 조건 설정

- 최대 길이를 256 토큰으로 제한하여 무한 루프 차단 및 안정적인 평가 환경 보장

3. 평가 메트릭 스토어 생성 (metrics)

- 데이터셋 전체의 통계 지표를 딕셔너리로 반환하여 모드별 비교를 위한 정량 데이터 확보

최종 지표를 산출하고 출력하여 모델링을 평가했어요

```
print("\n" + "=" * 50)
print("FINAL TEST RESULTS")
print("=" * 50)

print(f" Test CER           : {metrics['cer']:.4f} ({metrics['cer'] *
100:.2f}%)")
print(f" Test BLEU          : {metrics['bleu']:.4f}")
print(f" Exact Match Accuracy : {metrics['exact_match']:.4f}
({metrics['exact_match'] * 100:.2f}%)")

print("=" * 50)
```

출력

1. 가시성 확보를 위한 구분선 출력

- 문자열 연산("=" * 50)을 활용하여 터미널 콘솔창 내 독립적인 리포트 영역 시각화

2. 에러율 지표 퍼센트 변환 (metrics['cer'] * 100)

- 자문 오차율(CER) 소수점 데이터를 소수 둘째 자리 퍼센트(%) 단위로 포매팅 출력

3. 문맥 유사도 및 완전 정합도 표기

- n-gram 기반 문맥 복원 지표인 BLEU 스코어를 소수 넷째 자리까지 로깅
- 정답과 100% 일치한 비율인 Exact Match 지표를 동시 출력 →정량 평가 마무리

분석 결과 _ Analysis Results

이미지 전처리 모드별로 성능 지표를 대조했어요

original

```
=====
FINAL TEST RESULTS
=====
Test CER           : 0.1086 (10.86%)
Test BLEU          : 0.8507
Exact Match Accuracy : 0.3503 (35.03%)
=====
```

binarized _ otsu

```
=====
FINAL TEST RESULTS
=====
Test CER           : 0.1107 (11.07%)
Test BLEU          : 0.8402
Exact Match Accuracy : 0.3303 (33.03%)
=====
```

- 모두 BLEU 0.84 이상, Exact Match 33% 이상의 수식 복원 성능 기록
- 배경 잡음을 강제로 제거하는 �츠 이진화(binarized _ Otsu)보다, 원본 이미지(Original) 기반의 학습 결과가 모든 지표에서 미세하게 우세한 성능을 기록
- 원본 데이터셋 자체의 이미지 품질이 이미 충분히 깨끗하여 이진화 전처리의 실효성이 크지 않았음을 방증
- 오히려 이진화 프로세스(Otsu 알고리즘)를 거치면서 수식의 미세한 선 가닥이나 점 기호가 일부 손실되어 에러율(CER)이 0.21%p 미세하게 상승한 것으로 분석됨

가중치를 로드하여 샘플을 매칭했어요

```
encoder = FormulaDenseNetEncoder(hidden_dim=HIDDEN_DIM).to(DEVICE)
decoder = FormulaTransformerDecoder(vocab_size=len(vocab),
hidden_dim=HIDDEN_DIM).to(DEVICE)
```

1. 검증용 독립 모델 인스턴스 생성

- 전체 평가(Evaluation) 파이프라인과 별개로, 개별 샘플 문장을 한 줄씩 역디코딩하여 대조하기 위한 전용 인코더/디코더 객체 선언

2. 연산 하드웨어 가속 할당 (.to(DEVICE))

- 텐서 행렬 연산 병목을 제거, 고속 추론을 수행하기 위해 CPU에 있던 두 아키텍처구조를 T4 GPU(CUDA) 메모리로 탑재

```
load_checkpoint(
    BEST_CHECKPOINT,
    encoder,
    decoder,
    DEVICE,
)

encoder.eval()
decoder.eval()

decode_fn = make_decode_fn(vocab)
```

1. 최고 성능 가중치 로드 (load_checkpoint)

- 구글 드라이브에 저장된 최적 가중치(best.pt)를 모델 구조에 주입

2. 추론 최적화 환경 고정 (.eval())

- 드롭아웃(Dropout) 등 학습용 기능을 제외하여 평가 결과의 일관성 및 재현성 확보

3. 역토큰화 디코딩 함수 정의 (make_decode_fn)

- 모델이 출력한 숫자형 토큰 ID 배열을 사람이 읽을 수 있는 LaTeX 문자열로 변환

데이터를 추출하고 수식을 생성하여 샘플을 매칭했어요

```
batch = next(iter(test_loader))

images = batch["images"].to(DEVICE)
targets = batch["formula_ids"]

with torch.no_grad():
    memory = encoder(images)

    preds = decoder.generate(
        memory,
        sos_id=vocab.sos_id,
        eos_id=vocab.eos_id,
        max_len=256,
    )
```

1. 테스트 배치 로드 (next(iter(...)))

- 테스트 로더에서 첫 번째 이미지 및 정답 묶음을 추출하여 GPU 장치에 탑재

2. 추론 연산 최적화 (with torch.no_grad():)

- 가중치 업데이트가 필요 없는 구간의 메모리 버퍼를 제거하여 추론 속도 극대화

3. 자회귀적 수식 디코딩 (decoder.generate)

- 시작 토큰부터 끝 토큰 검출 시까지 이전 예측 토큰을 바탕으로 문맥을 결합·생성

데이터를 추출하고 수식을 생성하여 샘플을 매칭했어요

```
num_examples = min(5, images.size(0))

for i in range(num_examples):

    gt = decode_fn(targets[i])
    pred = decode_fn(preds[i])

    print("=" * 60)
    print(f"[Sample {i+1}]")
    print("GT      :", gt)
    print("PRED   :", pred)
    print("MATCH:", gt == pred)
    print("=" * 60)
```

1. 대표 샘플 제한 (min(5, ...))

- 오답 원인 분석을 위해 추출된 배치 내상위 5개 샘플에 대해서만 루프 가동

2. LaTeX 문자열 변환 (decode_fn)

- 정답과 예측값의 숫자형 토큰 ID 배열을 사람이 읽을 수 있는 LaTeX 문장으로 역변환

3. 기계적 일치 여부 판정 (gt == pred)

- 복원된 두 문자열의 공백 오차를 대조하여 콘솔에 MATCH: True/False 형태로 시각화 출력

분석 결과 _ Analysis Results

샘플1의 샘플 매칭 결과를 도출했어요

$$\alpha_1^r \gamma_1 + \cdots + \alpha_N^r \gamma_N = 0 \quad (r = 1, \dots, R)$$

원본 (sample)

- GT : $\alpha_{\{1\}}^{\{r\}} \gamma_{\{1\}} + \dots + \alpha_{\{N\}}^{\{r\}} \gamma_{\{N\}} = 0$
 $\forall (r = 1, \dots, R) \setminus ;$

$$\alpha_1^r \gamma_1 + \cdots + \alpha_N^r \gamma_N = 0 \quad (r = 1, \dots, R)$$

original

- PRED : $\alpha_{\{1\}}^{\{r\}} \gamma_{\{1\}} + \dots + \alpha_{\{N\}}^{\{r\}} \gamma_{\{N\}} = 0$
 $\forall (r = 1, \dots, R) \setminus ;$
- MATCH: False

$$\alpha_1^r \gamma_1 + \cdots + \alpha_N^r \gamma_N = 0 \quad (r = 1, \dots, R)$$

binarized _ otsu

- PRED : $\alpha_{\{1\}}^{\{r\}} \gamma_{\{1\}} + \dots + \alpha_{\{N\}}^{\{r\}} \gamma_{\{N\}} = 0$
 $\forall (r = 1, \dots, R) (\setminus ; \text{누락}) ,$
- MATCH: False

분석 결과 _ Analysis Results

샘플2의 샘플 매칭 결과를 도출했어요

$$\eta = -\frac{1}{2} \ln \left(\frac{\cosh \left(\sqrt{2} b_{\infty} \sqrt{1 + \alpha^2} y - \operatorname{arcsinh} \alpha \right)}{\sqrt{1 + \alpha^2}} \right)$$

$$\eta = -\frac{1}{2} \ln \left(\frac{\cosh \left(\sqrt{2} b_{\infty} \sqrt{1 + \alpha^2} y - \operatorname{arcsinh} \alpha \right)}{\sqrt{1 + \alpha^2}} \right)$$

$$\eta = -\frac{1}{2} \ln \left(\frac{\cosh \left(\sqrt{2} b_{\infty} \sqrt{1 + \alpha^2} y - \operatorname{arcsinh} \alpha \right)}{\sqrt{1 + \alpha^2}} \right)$$

원본 (sample)

- GT : $\eta = \dots \sqrt{1 + \alpha^2} y - \operatorname{arcsinh} \alpha$...

original

- PRED : $\eta = \dots \sqrt{1 + \alpha^2} y - \operatorname{arcsinh} \alpha$...
- MATCH: False

binarized _ otsu

- PRED : $\eta = \dots \sqrt{1 + \alpha^2} y - \operatorname{arcsinh} \alpha$...
- MATCH: False

분석 결과 _ Analysis Results

샘플3의 샘플 매칭 결과를 도출했어요

$$P_{(2)}^- = \int \beta d\beta d^9 p d^8 \lambda \Phi(-p, -\lambda) \left(-\frac{p^I p^I}{2\beta} \right) \Phi(p, \lambda)$$

$$P_{(2)}^- = \int \beta d\beta d\beta d^9 p d^8 \lambda \Phi(-p, -\lambda) \left(-\frac{p^I p^I}{2\beta} \right) \Phi(p, \lambda)$$

$$P_{(2)}^- = \int \beta d\rho d^0 p d^6 \Phi(-p, -\lambda) \left(-\frac{p^I p^I}{2\beta} \right) \Phi(p, \lambda)$$

원본 (sample)

- GT : $P_{(2)}^- = \int \beta d\beta d\beta d^9 p d^8 \lambda \Phi(-p, -\lambda) \left(-\frac{p^I p^I}{2\beta} \right) \Phi(p, \lambda)$

original

- PRED : $P_{(2)}^- = \int \beta d\beta d\beta d^9 p d^8 \lambda \Phi(-p, -\lambda) \left(-\frac{p^I p^I}{2\beta} \right) \Phi(p, \lambda)$...
- MATCH: False

binarized _ otsu

- PRED : $P_{(2)}^- = \int \beta d\rho d^0 p d^6 \Phi(-p, -\lambda) \left(-\frac{p^I p^I}{2\beta} \right) \Phi(p, \lambda)$...
- MATCH: False

분석 결과 _ Analysis Results

샘플4의 샘플 매칭 결과를 도출했어요

$$\Gamma(z+1) = \int_0^\infty dx e^{-x} x^z$$

원본 (sample)

- GT : $\mathbb{W}\text{Gamma}(z+1) = \mathbb{W}\text{int} _ \{0\}^{\mathbb{W}\text{infty}} \mathbb{W}, \mathbb{W}, dx \mathbb{W}, \mathbb{W}, e^{\{-x\}} x^{\{z\}}$

$$\Gamma(z+1) = \int_0^\infty dx e^{-x} x^z$$

original

- PRED : $\mathbb{W}\text{Gamma}(z+1) = \mathbb{W}\text{int} _ \{0\}^{\mathbb{W}\text{infty}} \sim \mathbf{d} \mathbf{x} \sim e^{\{-x\}} x^{\{z\}}$
- MATCH: False

$$\Gamma(z+1) = \int_0^\infty dx e^{-x} x^z$$

binarized _otsu

- PRED : $\mathbb{W}\text{Gamma}(z+1) = \mathbb{W}\text{int} _ \{0\}^{\mathbb{W}\text{infty}} (\mathbb{W}, \text{누락}) \mathbb{W}, dx \mathbb{W}, \mathbb{W}, e^{\{-x\}} x^{\{z\}}.$
- MATCH: False

분석 결과 _ Analysis Results

샘플5의 샘플 매칭 결과를 도출했어요

$$\frac{d}{ds} \mathbf{C}_i = \frac{1}{2} \epsilon_{ijk} \mathbf{C}_j \times \mathbf{C}_k$$

원본 (sample)

- GT : $\frac{d}{ds} \{\mathbf{C}\}_{-i} = \frac{1}{2} \epsilon_{ijk} \{\mathbf{C}\}_{-j} \times \{\mathbf{C}\}_{-k}, .$

$$\frac{d}{ds} \mathbf{C}_i = \frac{1}{2} \epsilon_{ijk} \mathbf{C}_j \times \mathbf{C}_k$$

original

- PRED : $\frac{d}{ds} \{\mathbf{C}\}_{-i} = \frac{1}{2} \epsilon_{ijk} \{\mathbf{C}\}_{-j} \times \{\mathbf{C}\}_{-k}, .$
- MATCH: True

$$\frac{d}{ds} \mathbf{C}_i = \frac{1}{2} \epsilon_{ijk} \mathbf{C}_j \times \mathbf{C}_k$$

binarized _otsu

- PRED : $\frac{d}{ds} \mathbf{C}_{-i} = \frac{1}{2} \epsilon_{ijk} \mathbf{C}_{-j} \times \mathbf{C}_{-k}, .$
- MATCH: False

사소한 문법 차이로 인한 매칭 실패 케이스를 분석했어요

1. 현상 : 출력 결과는 같으나 사소한 표현 차이 발생

- **말줄임표 중복 문법 (Sample 1)**
시각적으로 100% 동일한 기호를 출력하는
`\dots`와 `\ldots` 명령어 혼용
- **미세 서식 및 공백 혼선 (Sample 2 & 4)**
육안 구별이 불가능한 미세 공백(`\;` vs `\`, vs `~`) 및
함수 폰트 서식 토큰(`\mathrm` vs `\operatorname`) 불일치

2. 종합 분석 및 통계적 인사이트

- **Exact Match 지표의 맹점 규명**
실패 샘플 대부분이 수학적 의미와 맥락은 100% 완벽하게
복원
- **결론**
완전 정합도(35.03%)가 낮게 머문 원인은 모델의 성능
결함이 아닌, LaTeX 언어 고유의 중복 문법 표현으로 인해
평가지표가 극도로 보수적으로 집계되었기 때문

디코딩 한계 및 Original 전처리 우수성에 대한 유형을 분석했어요

1. 디코더 에러 패턴 대조 (sample 3)

- **Original 모드**
특정 토큰($d \setminus \beta$)을 중복 생성하는 루핑 에러 발생
→ 구조적 맥락을 즉시 복구하여 끝까지 수식 정합 완료
(견고함 입증)
- **binarized _ Otsu 모드**
장기 의존성 한계로 인해 수식 후반부 핵심 기호($\setminus \lambda$)
누락 → 문맥 붕괴

2. 전처리 최적화 최종 검증 (sample 5)

- **Original 단독 매칭 성공 (MATCH: True)**
수많은 대조군 중 오직 Original 학습 모드만 완전 정합에 성공
- **결론**
인위적인 이진화 처리(Otsu)보다 원본 이미지의 풍부한 픽셀
정보를 온전히 인코더에 전달하는 것이 디코더의 문맥 예측
정밀도를 높이는 데 결정적임을 방증

원본 그대로 학습하는 것이 가장 정확도가 높다는 결론을 도출했어요

1. 원본 데이터 학습의 우위

- 인위적으로 이미지를 흑백으로 변환한 방식(binarized _ Otsu)보다, 원본 이미지 그대로 학습시킨 방식(Original)이 모든 지표에서 더 우수한 정확도를 기록함
- 핵심 증거: 수많은 실패 샘플들 사이에서 오직 Original 모델만 유일하게 샘플 5번 수식을 완벽하게 맞추는 데 성공(MATCH: True)함

2. 사소한 차이로 인한 매칭 실패

- 샘플 1, 2, 4번처럼 수식의 수학적 의미와 맥락을 완벽하게 맞추었음에도 불구하고, 단지 눈에 보이지 않는 사소한 문법 차이(말줄임표 기호나 미세한 띄어쓰기 공백 명령어 차이) 때문에 잘못 출력되는 건들이 대다수 존재

3. 최종 결론

- 결론적으로 이진화 전처리는 오히려 미세한 수식 정보를 손실시켜 역효과를 내며, 원본 그대로 학습하는 것이 정확도가 높음
- (또한, 최종 매칭 정확도 수치(35.03%)가 낮게 나온 것은 모델의 성능 부족이 아니라 LaTeX 문법 특성상 기계적으로 엄격하게 감점당한 결과이며, 우리 모델의 실제 수식 복원 능력은 지표보다 훨씬 뛰어남)